

POLITÉCNICO DO PORTO

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

ROS/Gazebo

ISEP INSTITUTO SUPERIOR
DE ENGENHARIA DO PORTO DEPARTAMENTO DE ENGENHARIA

ELETROTÉCNICA
Instituto Superior de Engenharia do Porto

Maio, 2025

Índice

1	Robot Operating System (ROS)	1
1.1	O que é o Robot Operating System?	1
1.1.1	ROS Master	2
1.1.2	Nós	2
1.1.3	Tópicos	3
1.1.4	Mensagens	4
1.1.5	Serviços	5
1.2	Setup	5
1.2.1	Instalação do Ubuntu 20.04 e ROS	5
	WSL	6
	Distrobox	7
1.2.2	Inicialização do Workspace	8
1.3	Tutorial	9
1.3.1	Comandos Importantes	10
1.3.2	Criação de Pacotes	11
1.3.3	Estrutura de um Pacote	11
1.3.4	Publicador/Subscritor Simples	15
	C++	15
	Python	17
	Comunicação entre Publicador e Subscritor	19
1.3.5	Criação de Mensagens Personalizadas	20
	Criação do Ficheiro <code>.msg</code>	20
	Modificação dos ficheiros <code>package.xml</code> e <code>CMakeLists.txt</code>	21
	Exemplo de um Publicador para a Mensagem Personalizada	23
1.3.6	Serviços	24
	Criação do ficheiro <code>.srv</code>	24
	Modificação dos ficheiros <code>package.xml</code> e <code>CMakeLists.txt</code>	25
	Exemplo de um Nó Servidor	25
	Utilização do Comando <code>rosservice</code>	27
1.4	Exercícios Propostos	28
1.4.1	Setup para os Exercícios	28
1.4.2	Exercício 1 - Publicador e Subscritor	28

1.4.3	Exercício 2 - Mensagens Customizadas	29
1.4.4	Exercício 3 - Criação de um Serviço	29
2	Gazebo	31
2.1	O que é o Gazebo?	31
2.2	Integração do Gazebo com o ROS	31
2.3	Tutorial	32
3	Desafio	35
3.0.1	Introdução ao desafio	35
3.0.2	Instruções iniciais	35
	Referências	37

Capítulo 1

Robot Operating System (ROS)

Neste capítulo é apresentado o ROS, sendo feita uma breve apresentação sobre este software e os seus conceitos-chave. Também é explicado como se deve instalar o ROS, assim como o sistema operativo necessário, e como se cria um espaço de trabalho. Por fim, são explicados os conceitos mais importantes na criação de programas com o ROS.

1.1 O que é o Robot Operating System?

O *Robot Operating System* (ROS), apesar do seu nome, é um *middleware* (software que se encontra entre um sistema operativo e as aplicações que nele correm) para robótica, especificamente para o "desenvolvimento em larga escala de sistemas robóticos complexos". Foi desenvolvido em 2007 em Stanford durante o desenvolvimento do robô STAIR.[1]. Fornece um conjunto de bibliotecas, ferramentas e padrões que facilitam a criação de sistemas robóticos complexos e modulares[2].

Em 2010 foi lançada a primeira versão publica do ROS, à qual foi dada o nome Box Turtle. Desde esse lançamento e até hoje, o ROS manteve-se como um software opensource, que permitiu crescer a base de utilizadores.

Esta tecnologia é amplamente utilizada tanto em instituições académicas, no estudo da robótica ou em projetos de investigação [3], como na indústria, sendo utilizado por empresas como Fetch Robotics, Apex.AI e ClearPath. Também a Amazon Robotics utiliza o ROS nos seus robôs de movimentação de mercadorias e sistemas de automação logística. Estes robôs são projetados para otimizar a eficiência

e velocidade das operações de armazém, movendo autonomamente os itens para os funcionários que preenchem os pedidos.[4]

De modo a compreender melhor o ROS, vamos apresentar alguns dos conceitos básicos.

1.1.1 ROS Master

O ROS *Master* é o elemento central do sistema ROS, responsável por coordenar a comunicação entre os vários nós em execução. Sempre que cada nó é iniciado, este regista-se no *Master*, que mantém informação sobre os tópicos publicados e subscritos. Embora o *Master* não realize o transporte de mensagens, este elemento é essencial para permitir que os nós sejam capazes de estabelecer ligações entre si.

O ROS *Master* é o componente central do sistema ROS, responsável por coordenar a comunicação entre os vários nós. Sempre que um nó é iniciado, este regista-se no *Master*, que mantém informação sobre os tópicos publicados e subscritos. Embora o *Master* não transporte mensagens, é essencial para permitir que os nós se descubram mutuamente e estabeleçam ligações diretas entre si.

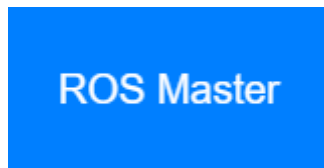


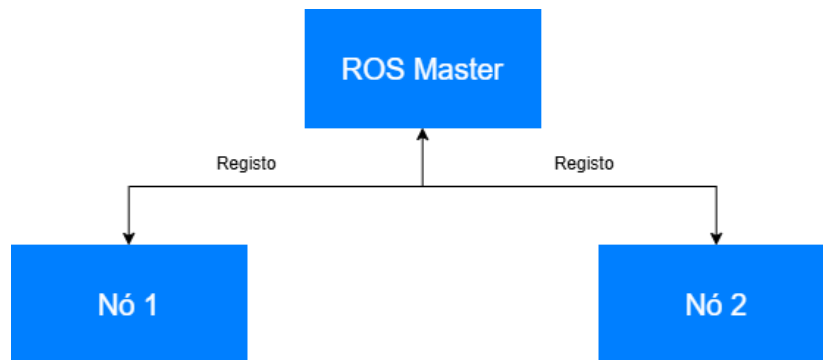
Figura 1.1: ROS *Master*

1.1.2 Nós

Os nós são os elementos funcionais do sistema ROS. São programas executáveis, normalmente escritos em C++ ou em Python. Estão organizado em pacotes ROS, podendo cada pacote conter um ou mais nós.

Os nós têm como objetivo cumprir um propósito único, ou seja, deve ser responsável por uma única função dentro do sistema. Esta divisão de funções permite, por exemplo, que diferentes nós sejam responsáveis pela leitura de sensores, controlo de atuadores, processamento de dados, entre outras tarefas. Esta divisão de tarefas promove a modularidade de sistemas, sendo este um dos objetivos do ROS.

A Figura 1.2 representa a ligação dos nós ao ROS *Master*, conforme descrito na Secção 1.1.1.

Figura 1.2: Ligação de Nós ao ROS *Master*

1.1.3 Tópicos

Os tópicos são o mecanismo utilizado pelos nós para comunicarem entre si no sistema ROS, através de um modelo de publicação/subscrição. Um nó pode publicar mensagens num determinado tópico, enquanto um ou mais nós podem subscrever esse mesmo tópico para receberem as mensagens transmitidas.

Cada tópico é identificado por um nome único no sistema, funcionando como elemento intermediário para o envio de mensagens entre nós, limitando-se a encaminhar as mensagens do publicador para todos os subscritores ativos no momento da publicação.

A figura 1.3 ilustra a ligação entre um nó publicador e um nó subscritor. Estes nós realizam o seu registo no ROS *Master*, podendo assim estabelecer uma ligação entre eles por meio do tópico. A partir do momento em que a ligação é estabelecida, as mensagens circulam do publicador para o(s) subscritor(es), sem intervenção do *Master*.

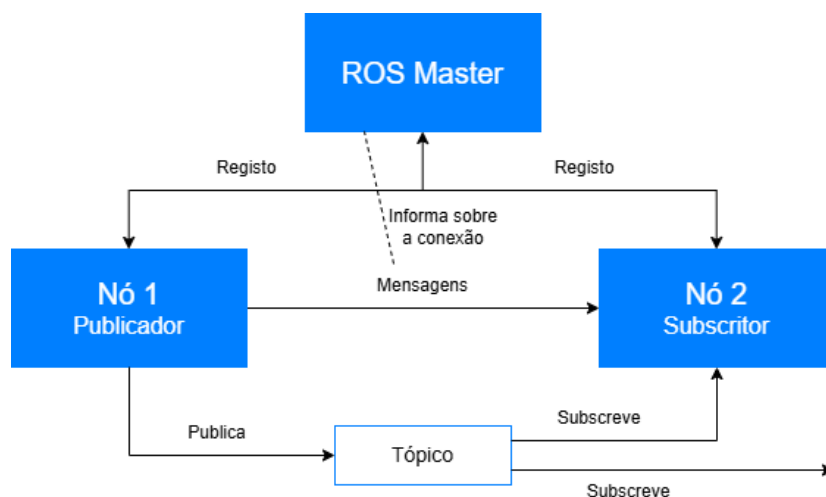


Figura 1.3: Comunicação entre nós ROS

1.1.4 Mensagens

As mensagens são estruturas de dados que definem o tipo de informação transmitida através de um tópico no sistema ROS. Podem assumir vários formatos, como inteiros, números de vírgula flutuante, strings, ou ainda combinações destes e de outros tipos. As mensagens personalizadas são normalmente definidas em ficheiros com a extensão `.msg`, localizados dentro do respetivo pacote ROS.

A figura 1.4 complementa a figura 1.3, ao demonstrar que cada tópico possui um tipo de mensagem bem definido. Esta definição garante que tanto o nó publicador como os subscritores partilham uma estrutura de dados comum, assegurando a integridade da comunicação.

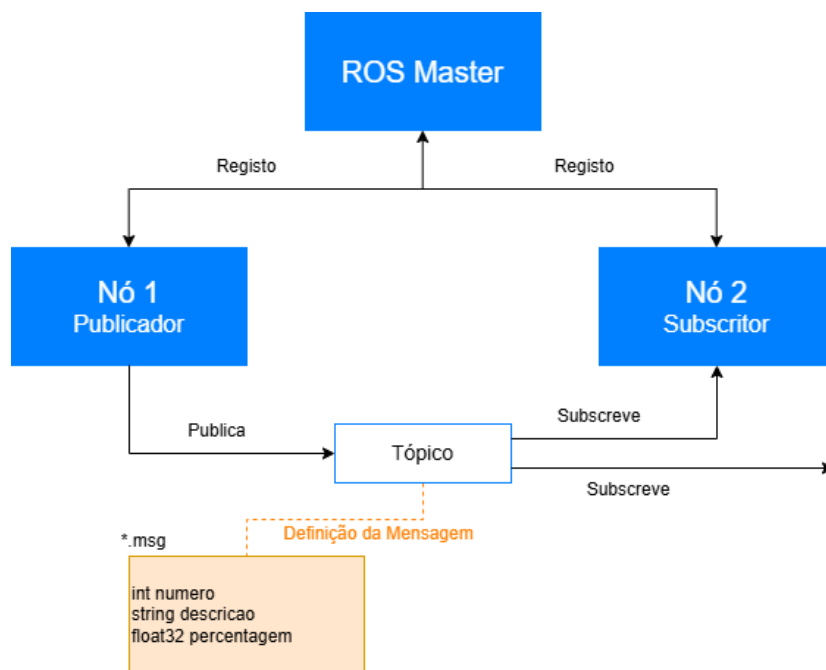


Figura 1.4: Comunicação entre nós ROS, com a definição do tipo de informação enviada

Como exemplo de definição de mensagem, apresenta-se abaixo a estrutura da mensagem `geometry_msgs/Point.msg`, incluída na biblioteca padrão do ROS. Esta mensagem define um ponto no espaço tridimensional através de três campos do tipo `float64`, correspondentes às coordenadas `x`, `y` e `z`.

```
geometry_msgs/Point.msg
```

```
float64 x
float64 y
float64 z
```

1.1.5 Serviços

Para a realização de comunicação baseada em pedido/resposta (*request/response*) entre nós, são utilizados serviços. Neste tipo de comunicação, existe um nó que atua como servidor, responsável por anunciar o serviço, e um nó cliente, que faz pedidos e recebe respostas.

De forma semelhante às mensagens, os serviços são definidos em ficheiros com a extensão `.srv`, localizados dentro do respetivo pacote ROS. Cada ficheiro `.srv` contém duas partes separadas por uma linha com três hífenos (`--`): a primeira especifica os campos do pedido (*request*) e a segunda os campos da resposta (*response*). É possível observar um exemplo genérico na figura 1.5 no bloco laranja.

A Figura 1.5 ilustra o funcionamento de um serviço entre um nó cliente e um nó servidor.

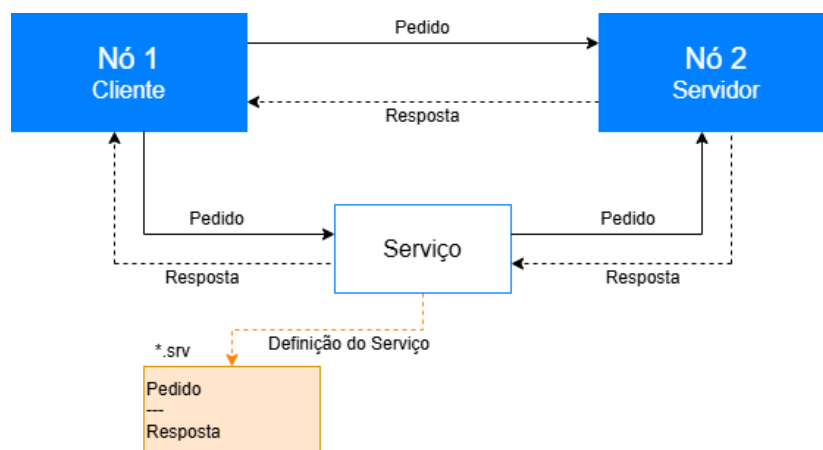


Figura 1.5: Comunicação entre nós ROS, por meio de serviços

Também é possível usufruir de serviços através da linha de comandos através do comando `rosservice`. Este será explorado mais à frente neste guião.

1.2 Setup

Nesta secção, é abordada a instalação do ROS e inicialização de um espaço de trabalho, assim como a instalação do Ubuntu 20.04 recorrendo ao *WSL*, para utilizadores *Windows*, ou recorrendo a uma *distrobox*, para utilizadores de *Linux* que não usem a distribuição necessária.

1.2.1 Instalação do Ubuntu 20.04 e ROS

Para este guião, vai ser usado o ROS *Noetic*, que é a versão do ROS para a versão 20.04 do Ubuntu. Para tal, é necessário instalar este sistema operativo, podendo fazê-lo em *Windows*, recorrendo ao *WSL*, ou então ao instalar esta distribuição no *Linux*, recorrendo a uma *distrobox*.

WSL

Para instalar o *Windows Subsystem for Linux* (WSL), é necessário abrir a *Windows Powershell* e executar o comando:

Instalação do WSL

```
wsl --install
```

Este comando instala o WSL, mais concretamente a versão 2. Esta versão do WSL permite a execução de aplicações que exibem interfaces gráficas.

Após instalar o WSL, é preciso instalar a distribuição Ubuntu 20.04 através da Microsoft Store

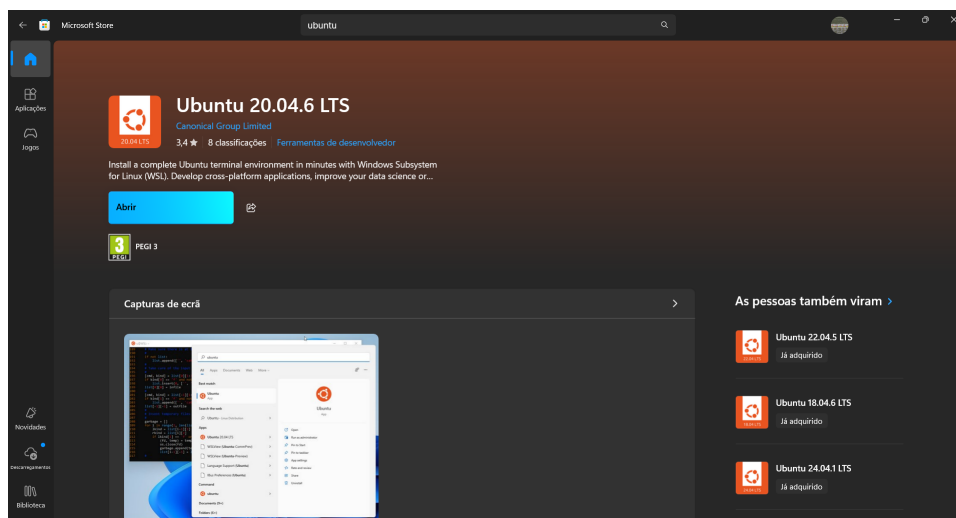


Figura 1.6: Ubuntu 20.04 na Microsoft Store.

Depois de instalado o Ubuntu 20.04, é só abrir um terminal, pesquisando na barra de procura por 'Ubuntu'.

Já com o terminal aberto, para instalar o ROS executa-se os seguintes comandos:

Instalação do ROS Noetic

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu
$(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'

sudo apt install curl

curl -s
https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo
apt-key add -

sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

Distrobox

A *Distrobox* permite utilizar qualquer distribuição do Linux a partir do terminal através do Docker (ou podman) através da criação de *containers* que usam a distribuição desejada. O *container* criado é integrado no sistema anfitrião, permitindo o acesso ao diretório home do utilizador, armazenamento e dispositivos externos, aplicações gráficas e áudio.

Para instalar a *distrobox*, primeiro precisamos de instalar o *Docker*, executando os seguintes comandos:

Instalação do Docker

```
# Remover Instalações Antigas
for pkg in docker.io docker-doc docker-compose podman-docker
containerd runc; do
    sudo apt-get remove $pkg
done

# Configura o repositório Docker
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update

# Instalar e Testar o Docker
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
sudo docker run hello-world

# Uso do Docker sem 'sudo'
```

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

Com o *Docker* instalado, procedemos à instalação da *Distrobox*:

Instalação da Distrobox

```
curl -s
https://raw.githubusercontent.com/89luca89/distrobox/main/install |
sudo sh
```

Com a *Distrobox* instalada, podemos agora instalar um container com uma distribuição *Ubuntu 20.04*:

Criação do Container com Ubuntu 20.04

```
distrobox create -i osrf/ros:noetic-desktop-full -n noetic
--additional-flags "--hostname <noetic-user>"
```

Com este comando, é criado um *container* chamado 'noetic' e definido o *hostname* do utilizador quando está dentro do container. É útil definir o *hostname* para poder automatizar alguns comandos recorrendo ao '.bashrc', como iremos ver mais à frente. Este comando, para além de criar um *container* com *Ubuntu 20.04*, também faz a instalação do ROS *Noetic*.

Para entrar dentro do *container*, basta executar o comando `distrobox enter noetic`

1.2.2 Inicialização do Workspace

Para inicializar o workspace, primeiro é necessário criar um diretório através do comando `mkdir catkin_ws` e depois navegar para dentro deste com `cd ~/catkin_ws`. Para criar o *workspace*, deve ser executado o comando `catkin build`, que apesar de termos instalado o ROS, este não vem incluído na instalação. Para instalar tal comando e inicializar o *workspace*, executamos os seguintes comandos:

Instalação do 'catkin build' e Inicialização do Workspace

```
# Instalação do catkin build
sudo sh \
-c 'echo "deb http://packages.ros.org/ros/ubuntu `lsb_release
-sc` main" \
> /etc/apt/sources.list.d/ros-latest.list'
```

```
wget http://packages.ros.org/ros.key -O - | sudo apt-key add -

sudo apt-get update
sudo apt-get install python3-catkin-tools

# Inicialização do workspace
mkdir catkin_ws
cd ~/catkin_ws
mkdir src
source /opt/ros/noetic/setup.bash
source /home/user/catkin_ws/devel/setup.bash
catkin build
```

Já com o *workspace* inicializado, para terminar, vamos editar o ficheiro `.bashrc` para que sejam feito o *source* do ROS e respetivo *workspace*. No ficheiro, devemos adicionar estas linhas:

```
.bashrc

## No final do documento adicionar estas linhas
# Para utilizadores com Distrobox
if [ "$HOSTNAME" = "<HOSTNAME Definido na Criação da Distrobox>" ];
then
    echo -e "\033[1;43mYou're in ROS Noetic!\e[0m"
    source /opt/ros/noetic/setup.bash
    source /home/user/catkin_ws/devel/setup.bash
fi

alias noetic='distrobox enter noetic'

# Para utilizadores com WSL
source /opt/ros/noetic/setup.bash
source /home/user/catkin_ws/devel/setup.bash
```

Ao ter estas linhas, sempre que seja aberto um novo terminal ou entrado na *distrobox*, o *source* do ROS é feito automaticamente. Adicionalmente, para os utilizadores da *distrobox*, criado um `alias` que permite que em vez de escrever o comando para aceder à *distrobox* (`distrobox enter noetic`), basta escrever `noetic`.

1.3 Tutorial

Nesta secção são abordados alguns comandos importantes do ROS, a criação de pacotes ROS e explicação do seu sistema de ficheiros, a criação de nós publicador e

subscritor e também a criação de mensagens e serviços personalizados.

1.3.1 Comandos Importantes

O ROS disponibiliza vários comandos no seu ambiente. Nesta secção do guião, serão apresentados os comandos mais importantes para executar nós, verificar o funcionamento do sistema, interagir com tópicos e serviços, bem como para diagnosticar o estado da rede ROS. Estes comandos serão fundamentais para a experimentação prática e a validação das aplicações desenvolvidas.

Como explicado na Secção 1.1.1, para que os nós possam estabelecer ligação entre si, é necessário que o ROS *Master* esteja em execução. Para iniciar o *Master*, deve ser utilizado o comando **roscore**, executado num terminal.

Com o ROS *Master*, podemos agora executar nós. Para tal, devemos executar o seguinte comando:

```
roslaunch nome_do_pacote nome_do_no
```

O ROS disponibiliza também comandos auxiliares para inspecionar ou interagir com os nós em execução. Seguem-se alguns exemplos:

```
# Comando para exibir a lista de nós em execução
roslaunch list

# Comando para exibir a informação de um dado nó
roslaunch info nome_do_no

# Comando para parar a execução de um nó
roslaunch kill nome_do_no
```

De uma forma semelhante ao **roslaunch**, também existe um comando para os tópicos chamado **rostopic**. Apresentam-se abaixo alguns dos seus usos mais comuns:

```
# Comando para exibir a lista de tópicos em execução
rostopic list

# Comando para exibir a informação de um dado tópico
rostopic info nome_do_topico

# Comando para exibir as mensagens publicadas num tópico
rostopic echo nome_do_no
```

Como referido na secção 1.1.5, existe ainda o comando `rosservice`, utilizado para interagir com serviços ROS:

```
# Comando para exibir a lista de serviços em execução
rosservice list

# Comando para exibir a informação de um dado serviço
rosservice info nome_do_servico

# Comando para chamar um serviço
rosservice call nome_do_servico argumentos
```

1.3.2 Criação de Pacotes

Para criar um pacote, devemos estar dentro do diretório `'src'` do *workspace*, onde podemos usar o comando `'catkin_create_pkg'`. Este comando tem como argumentos o nome do pacote a ser criado, assim como as dependências do pacote. Segue-se agora um exemplo de uso do `'catkin_create_pkg'`:

Criação de um Pacote

```
cd ~/catkin_ws/src
catkin_create_pkg tutorial_pkg roscpp rospy std_msgs sensor_msgs
```

Com este comando, foi criado um pacote chamado `'tutorial_pkg'` com as dependências `'roscpp'` e `'rospy'`, que permitem a escrita de programas em *C++* e *Python*, e também `'std_msgs'` e `'sensor_msgs'`, que são duas dependências que adicionam dois tipos de mensagens. Caso seja necessário adicionar mais dependências ao pacote, é possível ao alterar os ficheiros gerados na criação, como iremos ver a seguir.

1.3.3 Estrutura de um Pacote

Quando é criado um pacote, são gerados novos ficheiros e diretórios dentro do pacote, seguindo a estrutura representada na figura seguinte.

Normalmente, quando é criado um pacote, são criados os diretórios `'include'` e `'src'` e os ficheiros `'CMakeLists.txt'` e `'package.xml'`, sendo que o nosso foco estão nos ficheiros e no diretório `'src'`.

No ficheiro `'package.xml'` são definidas as dependências do pacote, como as necessárias para a compilação do projeto, para a execução do programa, para testes e para a geração de mensagens ou serviços personalizados. Segue-se o conteúdo do `'package.xml'` gerado quando foi criado o pacote `'tutorial_pkg'`:

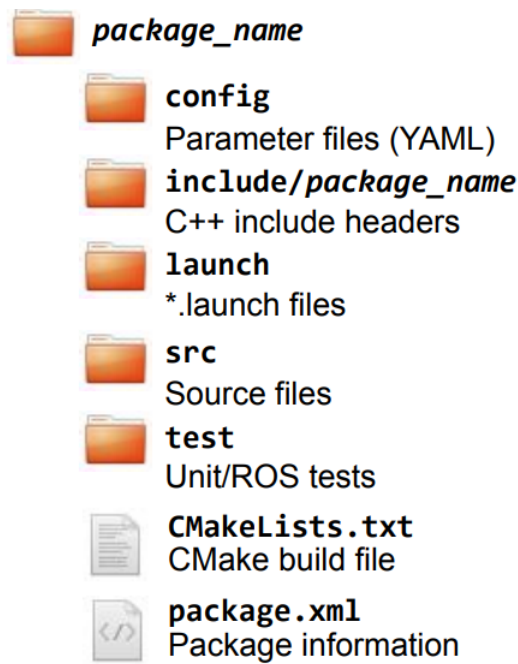


Figura 1.7: Estrutura de um Pacote.

package.xml

```

<?xml version="1.0"?>
<package format="2">
  <name>tutorial_pkg</name>
  <version>0.0.0</version>
  <description>The tutorial_pkg package</description>

  <maintainer email="jane.doe@example.com">Jane Doe</maintainer>

  <license>TODO</license>

  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>sensor_msgs</build_depend>
  <build_depend>std_msgs</build_depend>
  <build_export_depend>roscpp</build_export_depend>
  <build_export_depend>rospy</build_export_depend>
  <build_export_depend>sensor_msgs</build_export_depend>
  <build_export_depend>std_msgs</build_export_depend>
  <exec_depend>roscpp</exec_depend>
  <exec_depend>rospy</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>

```

```
<exec_depend>std_msgs</exec_depend>

<!-- The export tag contains other, unspecified, tags -->
<export>
  <!-- Other tools can request additional information be placed
  here -->

</export>
</package>
```

Ao analisar o conteúdo do ficheiro 'package.xml', observamos que este indica o nome e versão do pacote, uma descrição do pacote, o email e nome do *maintainer*, a licença utilizada e as dependências do projeto.

O ficheiro 'CMakeLists.txt' é responsável por definir as instruções de compilação do pacote. Nele são especificadas as versões mínimas do CMake e do ROS necessárias, os ficheiros fonte a compilar, as bibliotecas a ligar, e as dependências encontradas. Também define os alvos (executáveis ou bibliotecas) a serem construídos e a forma como estes interagem com outras partes do sistema ROS, como mensagens, serviços e nós. Normalmente, o 'CMakeLists.txt' gerado já possui um tutorial para indicar que comandos devem ser usados para uma determinada ação. Seguem-se os comandos mais usados com um exemplo de uso e uma breve explicação da sua funcionalidade.

CMakeLists.txt

```
## find_package -> Responsável por encontrar libraries e macros do
catkin
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  sensor_msgs
  std_msgs
)

## add_message_files -> Gera as mensagens definidas dentro do
diretório 'msg'. Este comando é semelhante para serviços e ações.
add_message_files(
  FILES
  Message1.msg
  Message2.msg
)
```

```
## add_executable -> Garante que os nós escritos em cpp possam ser
executados
add_executable(publisher_node src/publisher.cpp)
add_executable(listener_node src/listener.cpp)

## target_link_libraries -> Garante que são usadas as libraries
necessárias para a compilação
target_link_libraries(publisher_node
    ${catkin_LIBRARIES}
)
target_link_libraries(listener_node
    ${catkin_LIBRARIES}
)

## catkin_install_python -> Permite que os scripts python possam ser
executados como nós
catkin_install_python(PROGRAMS
    scripts/listener.py
    scripts/publisher.py
    scripts/helloWorld.py
    DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
```

O código dos nós criados para os pacotes estão definidos dentro do diretório 'src', caso a linguagem utilizada seja o C++. No caso do Python, devem estar dentro de um diretório chamado 'scripts'

Como pode ser observado na figura 1.7, podem existir mais alguns diretórios dentro de um pacote, tais como:

- 'config'- Armazena ficheiros de configuração, geralmente no formato .yaml, usados para definir parâmetros que os nós do ROS podem carregar usando o sistema de parâmetros.
- 'launch' - Contém ficheiro de "lançamento"(.launch) usados para iniciar um ou vários nós simultaneamente. São escritos em XML e permitem configurar tópicos, parâmetros, *namespaces*, entre outros.
- 'msg' - Contém definições de tipos de mensagens personalizadas que podem ser trocadas entre nós via tópicos. Os ficheiros .msg definem os campos da mensagem (tipo e nome), semelhantes a uma *struct*.
- 'srv' - Armazena definições de serviços personalizados (.srv), que são usados para comunicação síncrona entre nós (pedido e resposta). Cada ficheiro define dois blocos separados por —: um para o pedido e outro para a resposta.

1.3.4 Publicador/Subscritor Simples

Agora que temos o pacote criado, podemos proceder à escrita de dois nós simples: um publicador e um subscritor. Antes de abordar diretamente o código de cada um, começamos por um exemplo básico de um nó, de forma a introduzir as principais funções utilizadas. Como os programas podem ser escritos em C++ ou Python, este guião irá abordar ambas as linguagens.

C++

Para introduzir os elementos básicos de um nó ROS, apresenta-se de seguida um exemplo simples em C++ que imprime ciclicamente a string "Hello World".

helloWorld.cpp

```
#include <ros/ros.h>

int main(int argc, char* argv[])
{
    ros::init(argc, argv, "hello_world");
    ros::NodeHandle nh;
    ros::Rate loopRate(10);
    unsigned int count = 0;
    while (ros::ok()) {
        ROS_INFO_STREAM("Hello World " << count);
        ros::spinOnce();
        loopRate.sleep();
        count++;
    }
    return 0;
}
```

Este código começa por incluir o *header file* do ROS, permitindo aceder às funções necessárias para o funcionamento do nó. A função `ros::init(argc, argv, "hello_world")` inicializa o ROS e atribui o nome "hello_world" ao nó.

De seguida, é criado um *NodeHandle* público, que permite a comunicação entre o nó e o sistema ROS, declarado como `ros::NodeHandle nh`.

A taxa de execução do ciclo principal é definida através da função `ros::Rate loopRate(10)`, o que significa que o ciclo será executado 10 vezes por segundo (10 Hz).

O ciclo principal é executado enquanto o ROS estiver ativo, verificado pela função `ros::ok()`. Este ciclo termina caso o nó receba o sinal *SIGINT* (como um Ctrl+C) ou se for chamada a função `ros::shutdown()`.

Dentro do ciclo, a mensagem é enviada para o terminal com a função `ROS_INFO_STREAM`. Existem outras funções para registo de mensagens no terminal, como `ROS_INFO`, `ROS_WARN` e `ROS_ERROR`.

As últimas duas funções no ciclo, `ros::spinOnce()` e `loopRate.sleep()`, controlam o comportamento do nó: a primeira processa eventuais callbacks e a segunda assegura que o ciclo decorre à frequência definida.

Agora que as principais funções do ROS foram abordadas, segue-se a criação de um publicador e um subscritor simples.

publisher.cpp

```
#include <ros/ros.h>
#include <std_msgs/String.h>

int main(int argc, char **argv)
{
    ros::init(argc, argv, "talker");
    ros::NodeHandle nh;
    ros::Publisher stringPub =
        nh.advertise<std_msgs::String>("chatter", 1);
    ros::Rate loopRate(10);
    int count = 0;
    while(ros::ok())
    {
        std_msgs::String msg;
        msg.data = "Hello World - " + std::to_string(count);
        stringPub.publish(msg);
        ros::spinOnce();
        loopRate.sleep();
        count++;
    }
    return 0;
}
```

Este código cria um nó chamado "talker" que publica mensagens do tipo `std_msgs/String` no tópico "chatter". Esta funcionalidade é definida pela instrução `ros::Publisher stringPub = nh.advertise<std_msgs::String>("chatter", 1)`, onde são especificados o nome do tópico e o tamanho do *buffer* de mensagens.

Dentro do ciclo principal, é criada uma variável do tipo `std_msgs::String`, o seu conteúdo é atribuído com o contador `count`, e a mensagem é publicada com `stringPub.publish(msg)`.

Segue-se agora o código correspondente ao subscritor.

subscriber.cpp

```
#include <ros/ros.h>
#include <std_msgs/String.h>

void chatterCallback(const std_msgs::String& msg)
{
    ROS_INFO("I heard: [%s]", msg.data.c_str());
}

int main(int argc, char* argv[])
{
    ros::init(argc, argv, "listener");
    ros::NodeHandle nh;
    ros::Subscriber subscriber = nh.subscribe("chatter", 1,
        chatterCallback);
    ros::spin();
    return 0;
}
```

De forma semelhante ao publicador, este nó chamado "listener" é inicializado com `ros::init` e estabelece uma subscrição ao tópico "chatter" através da função `nh.subscribe("chatter", 1, chatterCallback)`. Sempre que uma nova mensagem é recebida, a função `chatterCallback` é automaticamente chamada, sendo responsável por imprimir o conteúdo da mensagem no terminal com `ROS_INFO`.

A chamada a `ros::spin()` mantém o nó em funcionamento, processando mensagens enquanto o ROS estiver ativo.

Python

Tal como foi feito em C++, apresenta-se agora um exemplo equivalente em Python que imprime ciclicamente a string "Hello World".

helloWorld.py

```
#!/usr/bin/env python3
import rospy

def main():
    rospy.init_node('hello_world')
    rate = rospy.Rate(10)
    count = 0
    while not rospy.is_shutdown():
        rospy.loginfo("Hello World %d", count)
```

```
rate.sleep()
count += 1

if __name__ == '__main__':
    main()
```

Este programa apresenta uma sintaxe mais simples do que a sua versão em C++, uma vez que não é necessário criar explicitamente um *NodeHandle*, nem utilizar a função `spinOnce()`. Apesar disso, o seu funcionamento é equivalente.

O nó é inicializado com o nome "hello_world" usando `rospy.init_node('hello_world')`, e a frequência de execução do ciclo é definida em 10 Hz com `rate = rospy.Rate(10)`. No interior do ciclo, a mensagem é escrita no terminal com `rospy.loginfo`, sendo o ciclo repetido enquanto o ROS estiver ativo.

Segue-se agora o código do publicador em Python.

```
publisher.py

#!/usr/bin/env python3
import rospy
from std_msgs.msg import String

def talker():
    rospy.init_node('talker')
    pub = rospy.Publisher('chatter', String, queue_size=1)
    rate = rospy.Rate(10)
    count = 0
    while not rospy.is_shutdown():
        msg = "Hello World - %d" % count
        rospy.loginfo(msg)
        pub.publish(msg)
        count += 1
        rate.sleep()

if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

Tal como o publicador em C++, este *script* cria um nó chamado "talker" que publica mensagens do tipo `std_msgs/String` no tópico "chatter". A publicação é feita com a função `rospy.Publisher`, onde são definidos o nome do tópico, o tipo da mensagem e o tamanho da fila.

Dentro do ciclo, a mensagem é criada e preenchida com o contador, escrita no terminal com `rospy.loginfo` e finalmente publicada com `pub.publish(msg)`. O ciclo continua enquanto o nó estiver ativo.

Para receber estas mensagens, é necessário um subscritor.

```
subscriber.py

#!/usr/bin/env python3
import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo("I heard: %s", data.data)

def listener():
    rospy.init_node('listener')
    rospy.Subscriber("chatter", String, callback)
    rospy.spin()

if __name__ == '__main__':
    listener()
```

Este *script* implementa um nó ROS em Python chamado "listener", que subscrive o tópico "chatter" e recebe mensagens do tipo `std_msgs/String`. A subscrição é criada com `rospy.Subscriber("chatter", String, callback)`, indicando que a função `callback` será chamada sempre que uma nova mensagem for recebida.

A função de *callback* imprime o conteúdo da mensagem no terminal com `rospy.loginfo`. A função `rospy.spin()` garante que o nó permanece ativo e continua a processar mensagens.

Comunicação entre Publicador e Subscritor

Depois de escritos os nós e devidamente configurados os ficheiros `CMakeLists.txt` e `package.xml` (como descrito na Secção 1.3.3), podemos compilar o pacote com `catkin build` e executá-lo via terminal. Para tal, utiliza-se o comando:

```
1 rosrun <nome_do_pacote> <nome_do_no>
```

Como exemplo, podemos executar o publicador e dois subscritores com nomes distintos, como "listener_cpp" e "listener_py", de forma a distingui-los.

Na Figura 1.8, observam-se dois subscritores a ler as mensagens publicadas por um publicador no tópico "chatter". Para visualizar a ligação entre os nós e tópicos,

```

[INFO] [1746107217.064032]: Hello World - 10376
[INFO] [1746107217.163927]: Hello World - 10377
[INFO] [1746107217.263957]: Hello World - 10378
[INFO] [1746107217.364081]: Hello World - 10379
[INFO] [1746107217.463688]: Hello World - 10380
[INFO] [1746107217.563478]: Hello World - 10381
[INFO] [1746107217.663481]: Hello World - 10382
[INFO] [1746107217.763451]: Hello World - 10383
[INFO] [1746107217.863608]: Hello World - 10384
[INFO] [1746107217.963478]: Hello World - 10385

[INFO] [1746107217.665398372]: I heard: [Hello World - 10382]
[INFO] [1746107217.765181941]: I heard: [Hello World - 10383]
[INFO] [1746107217.865862802]: I heard: [Hello World - 10384]
[INFO] [1746107217.965007953]: I heard: [Hello World - 10385]

matos@DESKTOP-OOLQIDU: ~/catkin_ws 71x5
[INFO] [1746107217.665384]: I heard: Hello World - 10382
[INFO] [1746107217.765184]: I heard: Hello World - 10383
[INFO] [1746107217.865937]: I heard: Hello World - 10384
[INFO] [1746107217.965128]: I heard: Hello World - 10385

```

Figura 1.8: Execução de um publicador (terminal da esquerda) e de dois subscritores (terminais da direita).

pode utilizar-se o comando `rqt_graph`, que gera um gráfico com os nós em execução e suas conexões.

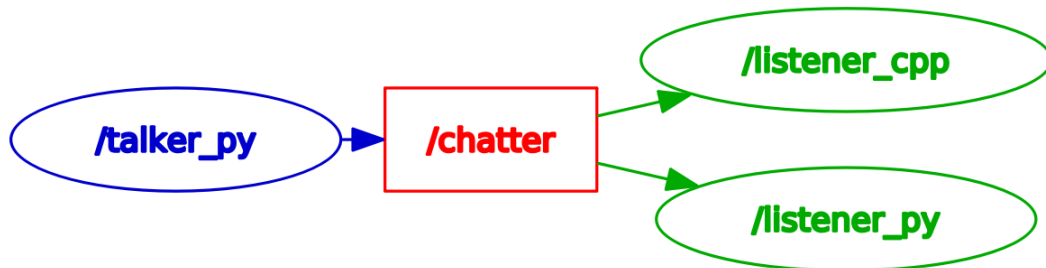


Figura 1.9: Output do comando `rqt_graph`.

1.3.5 Criação de Mensagens Personalizadas

Como referido na Secção 1.1.4, é possível definir mensagens personalizadas através da combinação de vários tipos de dados já existentes. Nesta parte do tutorial, demonstra-se como criar uma mensagem customizada com um ficheiro `.msg` e como compilar pacotes com esse tipo de mensagens, através da edição dos ficheiros `package.xml` e `CMakeLists.txt`. No final desta secção, apresenta-se também um exemplo de um nó publicador.

Criação do Ficheiro `.msg`

Antes de criar o ficheiro `.msg`, é necessário garantir que o pacote contém o diretório `msg`. Para o criar, pode-se utilizar o seguinte comando:

```
mkdir ~/catkin_ws/src/tutorial_pkg/msg
```

Com o diretório criado, o ficheiro de mensagem pode ser adicionado com:

```
touch ~/catkin_ws/src/tutorial_pkg/msg/tut_msg.msg
```

Em seguida, deve-se editar esse ficheiro com um editor de texto (como **gedit**, **vim**, **VSCode**, entre outros), e adicionar os seguintes campos:

```
tut_msg.msg

string id
int32 iteracao
sensor_msgs/Temperature temperatura
```

A mensagem **tut_msg** contém três campos: uma **string** chamada **id**, um campo **iteracao** do tipo **int32**, e um campo **temperatura** do tipo **sensor_msgs/Temperature**. Os dois primeiros campos utilizam tipos de dados básicos, mas o terceiro campo é baseado numa mensagem já existente no ROS.

Para consultar os campos de qualquer tipo de mensagem, pode-se utilizar o comando **rosmmsg info**. Por exemplo, ao executar:

```
Informação da mensagem sensor_msgs/Temperature

rosmmsg info sensor_msgs/Temperature
```

Obtém-se o seguinte resultado:

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
float64 temperature
float64 variance
```

Como se pode observar, esta mensagem inclui outros campos, entre os quais um campo do tipo **std_msgs/Header**, cuja estrutura também é revelada pelo comando acima.

Outra forma de consultar mensagens e os seus campos é através da wiki oficial do ROS, disponível em: wiki.ros.org.

Modificação dos ficheiros **package.xml** e **CMakeLists.txt**

Após a criação da mensagem, é necessário modificar os ficheiros **package.xml** e **CMakeLists.txt** para que o sistema consiga compilar corretamente o pacote.

No ficheiro **package.xml**, devem ser adicionadas as seguintes dependências:

package.xml

```
<build_depend>message_generation</build_depend>
<build_export_depend>message_generation</build_export_depend>
<exec_depend>message_runtime</exec_depend>
```

No CMakeLists.txt, devem ser feitas as seguintes alterações:

CMakeLists.txt

```
# Acrescentar 'message_generation' à lista de pacotes necessários
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  sensor_msgs
  std_msgs
  message_generation
)

add_message_files(
  FILES
  tut_msg.msg
)

generate_messages(
  DEPENDENCIES
  std_msgs sensor_msgs
)

catkin_package(
  CATKIN_DEPENDS roscpp rospy sensor_msgs std_msgs message_generation
)
```

É importante notar que o ficheiro CMakeLists.txt já contém estas secções, pelo que apenas será necessário adicionar dependências ou descomentar linhas conforme o necessário.

Após as alterações, o pacote pode ser compilado com o comando `catkin build`. Para verificar se a mensagem foi corretamente gerada, pode-se executar:

```
rosmmsg info tutorial_pkg/tut_msg
```

O resultado deverá ser semelhante ao seguinte:

```
string id
int32 iteracao
sensor_msgs/Temperature temperatura
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
float64 temperature
float64 variance
```

Exemplo de um Publicador para a Mensagem Personalizada

Como exemplo de utilização de uma mensagem personalizada num nó publicador, apresenta-se o seguinte código:

tut_msg_publisher.cpp

```
#include <ros/ros.h>
#include <tutorial_pkg/tut_msg.h>

int main(int argc, char **argv)
{
    ros::init(argc, argv, "custom_msg_talker");
    ros::NodeHandle nh;
    ros::Publisher customMsgPub =
    nh.advertise<tutorial_pkg::tut_msg>("custom_data", 1);
    ros::Rate loop_rate(10);

    int cnt = 0;

    while(ros::ok())
    {
        tutorial_pkg::tut_msg msg;

        // Preenchimento dos campos da mensagem principal
        msg.id = "Dados da Mensagem personalizada";
        msg.iteracao = cnt;

        // Preenchimento dos campos da submensagem 'temperatura'
        msg.temperatura.header.stamp = ros::Time::now();
        msg.temperatura.header.frame_id = "Dados de Temperatura
        Aleatorios";
        msg.temperatura.temperature = static_cast<float>(rand()) /
        static_cast<float>(RAND_MAX) * (27.0 - (-7.0)) + (-7.0);
```

```
// Publicação da mensagem
customMsgPub.publish(msg);
ROS_INFO("Mensagem publicada com sucesso!");

ros::spinOnce();
loop_rate.sleep();
cnt++;
}

return 0;
}
```

Este código apresenta um publicador que utiliza a mensagem personalizada criada anteriormente. Após incluir o cabeçalho da mensagem `tut_msg.h`, é definido um publicador para o tópico `custom_data`, com o tipo `tutorial_pkg::tut_msg`.

No interior do ciclo principal, a mensagem é preenchida com valores fixos ou gerados aleatoriamente. O campo `id` recebe uma `string`, `iteracao` um contador incremental, e o campo `temperatura` é preenchido com o tempo atual, um identificador de frame e uma temperatura aleatória entre $-7^{\circ}C$ e $27^{\circ}C$.

Após a preparação da mensagem, esta é publicada, e o ciclo repete-se enquanto o nó estiver em execução.

De lembrar que para executar este nó, devemos alterar o ficheiro `CMakeLists.txt` conforme abordado na secção 1.3.3

Este exemplo demonstra como criar e utilizar uma mensagem personalizada no ROS, desde a sua definição até à publicação num tópico. Este tipo de mensagens é particularmente útil quando se pretende transmitir dados estruturados e específicos entre nós, oferecendo maior flexibilidade e organização no desenvolvimento de sistemas mais complexos.

1.3.6 Serviços

Nesta secção é abordada a criação de um serviço através de um ficheiro `.srv`, bem como a edição dos ficheiros `package.xml` e `CMakeLists.txt`, e ainda a criação de um nó servidor para esse serviço. Será também demonstrada a utilização do comando `rosservice`. Neste exemplo, não é abordada a criação de um nó cliente.

Criação do ficheiro `.srv`

Antes de criar o ficheiro `.srv`, é necessário garantir que o pacote contém o diretório `srv`. Para o criar, pode-se utilizar o seguinte comando:

```
mkdir ~/catkin_ws/src/tutorial_pkg/srv
```

Para este exemplo, será criado um serviço que realiza a soma entre dois números inteiros. O ficheiro de definição do serviço pode ser criado com o comando:

```
touch ~/catkin_ws/src/tutorial_pkg/srv/AddTwoInts.srv
```

O conteúdo do ficheiro será:

AddTwoInts.srv

```
int64 Val1
int64 Val2
---
int64 Soma
```

Este ficheiro define que a requisição feita ao serviço incluirá dois inteiros de 64 bits chamados `Val1` e `Val2`, e que a resposta será um inteiro de 64 bits chamado `Soma`.

Modificação dos ficheiros `package.xml` e `CMakeLists.txt`

A modificação dos ficheiros de configuração é bastante semelhante à que foi feita para as mensagens personalizadas (ver Secção 1.3.5). A única diferença é a adição do bloco `add_service_files` no `CMakeLists.txt`:

CMakeLists.txt

```
add_service_files(
  FILES
  AddTwoInts.srv
)
```

Exemplo de um Nó Servidor

Para utilizar o serviço criado, é necessário desenvolver um nó que funcione como servidor. Seguem-se dois exemplos: um em `C++` e outro em `Python`. Depois de escrever o código, é necessário atualizar o `CMakeLists.txt` para compilar os novos nós (ver Secção 1.3.3).

Exemplo em `C++`:

server.cpp

```
#include <ros/ros.h>
#include <tutorial_pkg/AddTwoInts.h>

bool addTwoIntsCallback(tutorial_pkg::AddTwoInts::Request &req,
                        tutorial_pkg::AddTwoInts::Response &res)
{
    res.Soma = req.Val1 + req.Val2;
    ROS_INFO("Recebido: Val1 = %ld, Val2 = %ld | Soma = %ld",
              req.Val1, req.Val2, res.Soma);
    return true;
}

int main(int argc, char **argv)
{
    ros::init(argc, argv, "AddTwoInts_Server");
    ros::NodeHandle nh;

    ros::ServiceServer service = nh.advertiseService("AddTwoInts",
                                                      addTwoIntsCallback);
    ROS_INFO("Servidor pronto para somar 2 inteiros.");

    ros::spin();

    return 0;
}
```

Exemplo em Python:

server.py

```
#!/usr/bin/env python3
import rospy
from tutorial_pkg.srv import AddTwoInts, AddTwoIntsResponse

def addTwoIntsCallback(req):
    resultado = req.Val1 + req.Val2
    rospy.loginfo("Recebido: Val1=%d, Val2=%d | Resultado=%d",
                  req.Val1, req.Val2, resultado)
    return AddTwoIntsResponse(resultado)

def server():
    rospy.init_node('AddTwoInts_Server')
    serviceServer = rospy.Service('AddTwoInts', AddTwoInts,
                                   addTwoIntsCallback)
```

```
rospy.loginfo("Servidor pronto para somar 2 inteiros.")
rospy.spin()

if __name__ == '__main__':
    server()
```

Ambos os exemplos implementam um nó servidor que disponibiliza o serviço `AddTwoInts`. A função de *callback* (`addTwoIntsCallback`) é invocada sempre que o serviço é chamado. Esta função recebe os dois inteiros definidos na requisição do serviço (`Val1` e `Val2`), calcula a soma e devolve a resposta através do campo `Soma`.

No exemplo em C++, a função `ros::ServiceServer service = nh.advertiseService(...)` regista o serviço no nó, e `ros::spin()` mantém o processo ativo à espera de chamadas ao serviço. No exemplo em Python, o processo é equivalente, com a função `rospy.Service(...)` e o uso de `rospy.spin()`.

A principal diferença entre os dois exemplos é a sintaxe da linguagem, sendo a lógica do servidor idêntica. Ambos imprimem no terminal os valores recebidos e o resultado da operação com o auxílio da função `ROS_INFO` (em C++) ou `rospy.loginfo()` (em Python).

Utilização do Comando `rosservice`

Com o nó servidor em execução, podemos interagir com o serviço através do terminal, utilizando o comando `rosservice`. Para iniciar o servidor, podemos executar:

```
roslaunch tutorial_pkg <nome_do_executável>
```

ou, no caso do script Python:

```
roslaunch tutorial_pkg server.py
```

Com o servidor ativo, é possível chamar o serviço através do seguinte comando:

```
rosservice call /AddTwoInts "Val1: 0
Val2: 0"
```

Por exemplo, para somar os números 2 e 4, executa-se:

```
rosservice call /AddTwoInts "Val1: 2
Val2: 4"
```

O terminal deverá apresentar o seguinte resultado:

```
Soma: 6
```

Este exemplo demonstra como criar, configurar e utilizar um serviço simples no ROS, ilustrando o modelo de comunicação síncrona entre nós baseado em pedido e resposta.

1.4 Exercícios Propostos

Nesta secção são propostos alguns exercícios para que os tópicos abordados ao longo do tutorial possam ser consolidados. Começamos por criar um pacote onde os exercícios devem ser resolvidos e de seguida com a resolução dos mesmos.

1.4.1 Setup para os Exercícios

Para a realização dos exercícios, deve ser criado um novo pacote ROS. Para ser mais simples, devem usar o seguinte comando dentro do diretório `catkin_ws/src`:

```
catkin_create_pkg pkg_exercicios roscpp rospy sensor_msgs std_msgs  
geometry_msgs message_generation message_runtime
```

O uso deste comando faz com que o ficheiro `package.xml` já tenha quase todas as dependências necessárias aos exercícios, sendo apenas necessário adicionar a seguinte linha:

```
<build_export_depend>message_generation</build_export_depend>
```

Ao longo da realização dos exercícios devem modificar o ficheiro `CMakeLists.txt`, conforme explicado ao longo do tutorial para os vários exemplos.

1.4.2 Exercício 1 - Publicador e Subscritor

Neste exercício devem escrever o código, em C++ ou em Python, para três nós ROS:

- Um nó publicador para dois tópicos, um com o tipo de mensagem `sensor_msgs/Temperature` e outro com o tipo de mensagem `geometry_msgs/PointStamped`. Devem preencher as mensagens com valores aleatório recorrendo à função `rand()`. Caso seja necessário preencher algum campo do tipo `String`, este deve ser preenchido com uma `String` que identifique o tipo de mensagem enviado.

- Dois nós, em que cada um deve receber as mensagens publicadas num dos tópicos

Após a implementação dos nós, deve verificar o comportamento da comunicação entre eles utilizando a ferramenta `rqt_graph`, para visualizar graficamente os tópicos e ligações existentes.

Dica

Para saber o conteúdo de cada tipo de mensagem a utilizar, usar o comando `rosmmsg info nome_da_mensagem` ou consultar os seguintes links:

- https://docs.ros.org/en/noetic/api/sensor_msgs/html/index-msg.html
- https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/PointStamped.html

1.4.3 Exercício 2 - Mensagens Customizadas

Para este exercício, deve ser criada uma mensagem customizada com os seguintes campos:

- Um campo do tipo `String` chamado "msgHeader"
- Um campo do tipo `sensor_msgs/Temperature` chamado "temperatura"
- Um campo do tipo `geometry_msgs/PointStamped` chamado "posicao"

Após a definição da mensagem, escrever o código para um nó publicador recorrendo ao `C++` ou ao `Python` para a publicação de dados aleatórios recorrendo à função `rand()`. Caso seja necessário preencher algum campo do tipo `String`, este deve ser preenchido com uma `String` que identifique o tipo de mensagem enviado.

Depois de escrito o nó, deve executá-lo e verificar a publicação dos dados com o comando `rostopic echo nome_do_topico`.

1.4.4 Exercício 3 - Criação de um Serviço

O objetivo deste exercício é a implementação de um servidor multiserviço. Para tal, devem ser criados dois serviços:

- Um serviço para fazer a conversão de graus para radianos
- Um serviço para fazer a conversão de radianos para graus

Os parâmetros dos serviços devem ser do tipo `float64`. Depois de definidos os serviços, deve ser escrito o programa `C++` ou `Python` para o nó servidor, executar este nó e chamar os serviços recorrendo ao comando `rosservice call`.

Dica

Para fazer as conversões, seguem-se as formulas a usar:

- $graus = \frac{(radianos * 180)}{\pi}$
- $radianos = \frac{(graus * \pi)}{180}$

Capítulo 2

Gazebo

2.1 O que é o Gazebo?

O *Gazebo* é um simulador de robótica *open source* que surgiu com o objetivo de fornecer uma plataforma realista para testar algoritmos de controlo, visão computacional e comportamento de robôs em ambientes complexos, sem os riscos e custos associados à experimentação no mundo real. Desenvolvido inicialmente no âmbito do *Player Project* por Nathan Koenig e colegas, o *Gazebo* ganhou notoriedade por permitir simulações 3D altamente detalhadas e fisicamente realistas, recorrendo a motores de física como *ODE*, *Bullet* e *DART* [5]. Com o tempo, passou a ser mantido pela Open Source Robotics Foundation (OSRF), tornando-se uma ferramenta central no ecossistema do ROS [6][7].

Uma das principais vantagens do *Gazebo* é a sua capacidade de simular ambientes tridimensionais com elevado realismo quer ao nível gráfico, quer ao nível de físicas dos robôs, permitindo a utilização de múltiplos modelos de sensores virtuais, são exemplo: câmaras RGB-D, LiDARs e GPS. Assim sendo o *Gazebo* apresenta-se com uma plataforma ideal para prototipagem e validação de algoritmos de navegação e perceção [8].

2.2 Integração do Gazebo com o ROS

A integração entre o *Gazebo* e o ROS constitui uma das principais razões para a ampla adoção de ambas as ferramentas no desenvolvimento de sistemas robóticos

modernos. Esta interligação é viabilizada por um conjunto de *plugins* e pacotes dedicados, como o `gazebo_ros_pkgs`, que permitem a comunicação entre o ambiente de simulação e os nós ROS [9]. Através desta interface, é possível publicar e subscrever tópicos, assim como utilizar serviços e ações. Outra particularidade interessante é a possibilidade de simular sensores e atuadores com grande fidelidade. Por exemplo, sensores simulados como câmaras, LiDARs ou IMUs geram mensagens ROS compatíveis (como `sensor_msgs/Image` ou `sensor_msgs/LaserScan`), que podem ser processadas pelos mais diversos algoritmos de perceção ou outros necessários as tarefas dos robôs [10]. Da mesma forma, comandos de controlo podem ser enviados para atuadores simulados utilizando mensagens padronizadas (como `geometry_msgs/Twist`), o que garante uma aproximação entre o desenvolvimento em simulação e a aplicação em robôs reais [11].

2.3 Tutorial

O *TurtleBot3* é uma plataforma robótica modular, de baixo custo, desenvolvida pela *ROBOTIS* em colaboração com a Open Source Robotics Foundation (OSRF), concebida para facilitar o ensino, a investigação e o desenvolvimento de aplicações em robótica com o ROS. Graças à sua compatibilidade total com o ecossistema ROS e à integração nativa com o simulador Gazebo, o TurtleBot3 é amplamente utilizado em ambientes académicos e laboratoriais para a experimentação com algoritmos de navegação, SLAM (Simultaneous Localization and Mapping), controlo e perceção. Com suporte para diversos sensores (como LiDAR, IMU e encoders de rodas) e modelos distintos (Burger, Waffle Pi, etc.), a plataforma oferece uma excelente relação entre simplicidade e funcionalidade, permitindo ao utilizador desenvolver e testar soluções tanto em simulação como em hardware real com mínima adaptação de código. Este capítulo tem a intenção de servir como um pequeno tutorial para a instalação deste robô no ROS e como posso ser usado em conjunto com o simulador *Gazebo*.

Para realizar a simulação do *TurtleBot3* no *Gazebo*, é necessário garantir que os pacotes estão devidamente instalados. A instalação pode ser feita com os seguintes comandos:

```
sudo apt update
sudo apt install ros-<distro>-turtlebot3
```

Substituindo `<distro>` pela versão do ROS instalada (como `noetic` ou `melodic`). Após a instalação, deve-se definir o modelo do robô a ser utilizado através da variável de ambiente `TURTLEBOT3_MODEL`. Por exemplo, para o modelo `burger`, o comando é:

```
export TURTLEBOT3_MODEL=burger
```

Para que esta variável seja carregada automaticamente em todos os terminais, pode-se adicioná-la ao final do ficheiro `.bashrc` com o seguinte conjunto de comandos:

```
echo "export TURTLEBOT3_MODEL=burger" >> ~/.bashrc  
source ~/.bashrc
```

Com o ambiente configurado, a simulação é iniciada com:

```
roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

Este comando abre o Gazebo com o robô num ambiente virtual. Para controlar o TurtleBot3 manualmente, pode-se utilizar o teclado com:

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

A interação com o robô será então possível usando as teclas do teclado. Para encerrar a simulação, basta pressionar `Ctrl+C` no terminal onde o `roslaunch` está a correr. Adicionalmente, é possível experimentar outros cenários substituindo o ficheiro `.launch`, como:

```
turtlebot3_empty_world.launch  
turtlebot3_house.launch
```


Capítulo 3

Desafio

3.0.1 Introdução ao desafio

Neste capítulo é apresentado o desafio final, cujo principal objetivo é aplicar, de forma integrada, os conhecimentos adquiridos ao longo do trabalho. O desafio proposto consiste em desenvolver uma aplicação robótica baseada em ROS e simulada no ambiente *Gazebo*, utilizando o robô *TurtleBot3 Waffle*. A tarefa passa por criar um nó que permita ao robô deslocar-se autonomamente dentro do mundo virtual `turtlebot3_world`, evitando obstáculos durante a navegação. Inspirado no funcionamento de um robô aspirador, é proposto a criação de código para utilizar a informação providenciada por um *Laser Range Finder* e fazer o controlo dos atuadores do robô de modo a que o mesmo se desvie de obstáculos do ambiente onde o mesmo se encontra.

3.0.2 Instruções iniciais

Para dar início ao desafio, é necessário garantir que o pacote de trabalho se encontra disponível no ambiente de desenvolvimento. Caso ainda não tenha sido feito, o leitor deve clonar o repositório disponível em: https://github.com/A-M4t0s/formacao_ros_gazebo.git, onde se encontra o diretório `ex_final_pkg`, já preparado com a estrutura base do projeto. Este pacote inclui uma implementação parcial de um nó ROS em *Python*, localizado em `scripts/aspirador.py`, e encontra-se organizado com diretórios padrão de um pacote ROS, como `launch/`, `src/` e `scripts/`. O ficheiro

principal do desafio, localizado em `scripts/aspirador.py`, já contém definições básicas, comentários e blocos de código comentados, bem como várias marcações com `TODO`, que indicam os pontos onde o utilizador deverá intervir. Em alternativa, também está disponível um esqueleto equivalente em `src/aspirador.cpp`, permitindo que o desafio seja realizado quer em *Python*, quer em *C++*, conforme a preferência do utilizador. Em particular, está pendente a implementação da função de callback para receção de mensagens do tipo `sensor_msgs/LaserScan`, a criação de um publicador no tópico `/cmd_vel` do tipo `geometry_msgs/Twist`, um **subscritor no tópico `/scan`**, bem como a definição da lógica de controlo responsável por movimentar o robô e evitar obstáculos. O desafio, portanto, consiste em completar estas partes em falta, utilizando os blocos já definidos como base.

Referências

- [1] A. Koubaa, *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Cham: Springer International Publishing, 2016. [Citado na página 1]
- [2] ROS.org, “About ros,” 2025. Accessed: 2025-04-30. [Citado na página 1]
- [3] ROS, “Ros: Why ros?,” 2024. Accessed: 2025-04-30. [Citado na página 1]
- [4] AWS Robotics Blog, “Deploy and manage ros robots with aws iot greengrass 2.0 and docker,” 2022. [Citado na página 2]
- [5] N. Koenig and A. Howard, “Design and use paradigms for gazebo, an open-source multi-robot simulator,” Tech. Rep. CMU-RI-TR-04-34, University of Southern California, 2004. [Citado na página 31]
- [6] O. S. R. Foundation, “Gazebo simulator,” 2025. [Citado na página 31]
- [7] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “Ros: an open-source robot operating system,” Tech. Rep. SAIL-T-2009-002, Stanford Artificial Intelligence Laboratory, 2009. [Citado na página 31]
- [8] G. Project, “Sensors in gazebo,” 2025. [Citado na página 31]
- [9] O. S. R. Foundation, “gazebo_ros_pkgs,” 2025. [Citado na página 32]
- [10] R. Wiki, “sensor_msgs - standard sensor messages,” 2025. [Citado na página 32]
- [11] R. Wiki, “geometry_msgs - common geometry messages,” 2025. [Citado na página 32]